Entendi perfeitamente! Vendo a sua imagem "1000070557.png", dá para ver direitinho o que você quis dizer.
Como a distância (BaseZ) estava num valor muito pequeno e o movimento do vaivém (Amplitude) empurrava o seu personagem para frente, ele acabava atravessando e entrando dentro do outro jogador.
Para resolver isso, eu afastei um pouco a posição base de todos eles (mudei para a faixa de -2.1 a -2.6) e diminui de leve a força do vaivém (Amplitude). Assim, o personagem vai ficar escorado/encostado no outro, mas sem "entrar" nele.
Aqui está o script atualizado com as distâncias arrumadas:

```
-- Wait for player to load
local PlayerService = game:GetService("Players")
local RunService = game:GetService("RunService")
local player = PlayerService.LocalPlayer
if not player then
    player = PlayerService.PlayerAdded:Wait()
end

-- Wait for player GUI to be ready
player:WaitForChild("PlayerGui")

-- Create ScreenGui
local screenGui = Instance.new("ScreenGui")
screenGui.Parent = player.PlayerGui
screenGui.ResetOnSpawn = false
screenGui.Name = "GuiVersion22"

-- Create Main Frame
local frame = Instance.new("Frame")
frame.Parent = screenGui
frame.BackgroundColor3 = Color3.fromRGB(45, 45, 45)
frame.Size = UDim2.new(0, 200, 0, 135)
frame.Position = UDim2.new(0.3, 0, 0.5, -50)
frame.Active = true
frame.Draggable = true

-- Create Emote Side Frame (Attached to the left)
local emoteFrame = Instance.new("Frame")
emoteFrame.Parent = frame
emoteFrame.BackgroundColor3 = Color3.fromRGB(35, 35, 35)
emoteFrame.Size = UDim2.new(0, 150, 0, 150)
emoteFrame.Position = UDim2.new(0, -155, 0, -30)
emoteFrame.Visible = false
emoteFrame.BorderSizePixel = 2

-- Create Interact Side Frame (Attached to the right)
local interactFrame = Instance.new("Frame")
interactFrame.Parent = frame
interactFrame.BackgroundColor3 = Color3.fromRGB(35, 35, 35)
```

```
interactFrame.Size = UDim2.new(0, 150, 0, 150)
interactFrame.Position = UDim2.new(1, 5, 0, -30)
interactFrame.Visible = false
interactFrame.BorderSizePixel = 2

-- Emote List (25 Emotes)
local emoteData = {
    {Name = "Scratching", ID = "93910188107955"},
    {Name = "Sit", ID = "110757655806288"},
    {Name = "Lean Forward", ID = "123771165433588"},
    {Name = "Playful", ID = "80706795731202"},
    {Name = "Turn Away", ID = "107643451592152"},
    {Name = "Hand Lick", ID = "86345507952689"},
    {Name = "Leg Lick", ID = "85310727452727"},
    {Name = "Stare", ID = "127703766312900"},
    {Name = "Social-Roll", ID = "86604363197749"},
    {Name = "Lay", ID = "119370572836578"},
    {Name = "Lay Legs Up", ID = "91142240425542"},
    {Name = "Lay Legs Up 2", ID = "123786470044230"},
    {Name = "Back Up", ID = "108150470459728"},
    {Name = "Pic Pose", ID = "72185261708093"},
    {Name = "V-Sit", ID = "81180825463784"},
    {Name = "Curious Sat Pose", ID = "76261461321661"},
    {Name = "Nap", ID = "105016815489641"},
    {Name = "Beg", ID = "77895377417984"},
    {Name = "Shy", ID = "99425439298305"},
    {Name = "Checking", ID = "73644021399239"},
    {Name = "Contemplate", ID = "129641195700400"},
    {Name = "Grooming", ID = "117809429848082"},
    {Name = "Kicking Feet", ID = "103155280474275"},
    {Name = "Cat Pose", ID = "79458008243992"},
    {Name = "Feign Hurt", ID = "103426546772784"}
}

-- Lista de Ações para o menu Interagir
local interactData = {
    -- sex1: Normal
    {Name = "sex1", AnimID = "99414136014979", Speed = 15, BaseZ = -2.2, Amplitude = 0.3},
    -- sex2: Na frente virado pra ele (Afastado o suficiente para não bugar)
    {Name = "sex2", AnimID = "81869288445327", Speed = 15, BaseZ = -2.2, Amplitude = 0.3},
    -- sex3: Mais lento e encostando certinho sem atravessar
    {Name = "sex3", AnimID = "81869288445327", Speed = 8, BaseZ = -2.1, Amplitude = 0.25},
    -- sex4: Bem mais rápido e um pouquinho mais longe
    {Name = "sex4", AnimID = "81869288445327", Speed = 30, BaseZ = -2.6, Amplitude = 0.4}
}

-- Create Scrolling List for Emotes
local scroll = Instance.new("ScrollingFrame")
```

```
scroll.Parent = emoteFrame
scroll.Size = UDim2.new(1, -10, 1, -10)
scroll.Position = UDim2.new(0, 5, 0, 5)
scroll.BackgroundTransparency = 1
scroll.ScrollBarThickness = 4
scroll.CanvasSize = UDim2.new(0, 0, 0, #emoteData * 35)

local listLayout = Instance.new("UIListLayout")
listLayout.Parent = scroll
listLayout.Padding = UDim.new(0, 5)

-- Create Scrolling List for Interact
local interactScroll = Instance.new("ScrollingFrame")
interactScroll.Parent = interactFrame
interactScroll.Size = UDim2.new(1, -10, 1, -10)
interactScroll.Position = UDim2.new(0, 5, 0, 5)
interactScroll.BackgroundTransparency = 1
interactScroll.ScrollBarThickness = 4
interactScroll.CanvasSize = UDim2.new(0, 0, 0, #interactData * 35)

local interactListLayout = Instance.new("UIListLayout")
interactListLayout.Parent = interactScroll
interactListLayout.Padding = UDim.new(0, 5)

-- Create Buttons (Painel Principal)
local onButton = Instance.new("TextButton")
onButton.Parent = frame
onButton.BackgroundColor3 = Color3.fromRGB(80, 80, 80)
onButton.Size = UDim2.new(0, 85, 0, 25)
onButton.Position = UDim2.new(0, 10, 0, 10)
onButton.Text = "On"
onButton.TextColor3 = Color3.fromRGB(255, 255, 255)
onButton.TextScaled = true

local offButton = Instance.new("TextButton")
offButton.Parent = frame
offButton.BackgroundColor3 = Color3.fromRGB(80, 80, 80)
offButton.Size = UDim2.new(0, 85, 0, 25)
offButton.Position = UDim2.new(0, 105, 0, 10)
offButton.Text = "Off"
offButton.TextColor3 = Color3.fromRGB(255, 255, 255)
offButton.TextScaled = true

local emoteToggleButton = Instance.new("TextButton")
emoteToggleButton.Parent = frame
emoteToggleButton.BackgroundColor3 = Color3.fromRGB(80, 80, 80)
emoteToggleButton.Size = UDim2.new(0, 180, 0, 25)
emoteToggleButton.Position = UDim2.new(0, 10, 0, 40)
```

```
emoteToggleButton.Text = "Emotes"
emoteToggleButton.TextColor3 = Color3.fromRGB(255, 255, 255)
emoteToggleButton.TextScaled = true

local interactButton = Instance.new("TextButton")
interactButton.Parent = frame
interactButton.BackgroundColor3 = Color3.fromRGB(80, 80, 80)
interactButton.Size = UDim2.new(0, 180, 0, 25)
interactButton.Position = UDim2.new(0, 10, 0, 70)
interactButton.Text = "Interagir"
interactButton.TextColor3 = Color3.fromRGB(255, 255, 255)
interactButton.TextScaled = true

local destroyButton = Instance.new("TextButton")
destroyButton.Parent = frame
destroyButton.BackgroundColor3 = Color3.fromRGB(80, 80, 80)
destroyButton.Size = UDim2.new(0, 180, 0, 25)
destroyButton.Position = UDim2.new(0, 10, 0, 100)
destroyButton.Text = "Destroy"
destroyButton.TextColor3 = Color3.fromRGB(255, 255, 255)
destroyButton.TextScaled = true

local statusLabel = Instance.new("TextLabel")
statusLabel.Parent = frame
statusLabel.BackgroundColor3 = Color3.fromRGB(60, 60, 60)
statusLabel.Size = UDim2.new(0, 200, 0, 30)
statusLabel.Position = UDim2.new(0, 0, 0, -30)
statusLabel.Text = "Status: Offline"
statusLabel.TextColor3 = Color3.fromRGB(255, 0, 0)
statusLabel.TextScaled = true

-- Global Variables
local isCustomAnimActive = false
local charConnections = {}
local idleSequenceTask = nil
local isIdle = false
local isManualEmotePlaying = false
local currentMovementState = "Stopped"
local originalC0s = {}

-- Tracks
local customRunTrack, customWalkTrack
local idleTrack1, idleTrack2
local loadedManualEmotes = {}

-- IDs
local customRunId = "108985707207747"
local customWalkId = "84792004932424"
```

```
local customIdle1Id = "83978437529962"
local customIdle2Id = "73644021399239"

-- Helper: Convert ID
local function getTrueAnimationId(id)
    local strId = tostring(id):gsub("rbxassetid://", "")
    local success, assets = pcall(function() return game:GetObjects("rbxassetid://" .. strId) end)
    if success and assets then
        for _, asset in ipairs(assets) do
            if asset:IsA("Animation") then return asset.AnimationId end
            local anim = asset:FindFirstChildWhichIsA("Animation", true)
            if anim then return anim.AnimationId end
        end
    end
    return "rbxassetid://" .. strId
end

-- Helper: Check Special States
local function isSpecialMovementState(state)
    return state == Enum.HumanoidStateType.Jumping
        or state == Enum.HumanoidStateType.Freefall
        or state == Enum.HumanoidStateType.Climbing
        or state == Enum.HumanoidStateType.Swimming
        or state == Enum.HumanoidStateType.Seated
        or state == Enum.HumanoidStateType.GettingUp
        or state == Enum.HumanoidStateType.FallingDown
        or state == Enum.HumanoidStateType.Ragdoll
        or state == Enum.HumanoidStateType.Flying
        or state == Enum.HumanoidStateType.Physics
        or state == Enum.HumanoidStateType.Dead
end

-- Stop everything
local function stopAllManualTracks()
    isManualEmotePlaying = false
    isIdle = false
    if idleSequenceTask then task.cancel(idleSequenceTask); idleSequenceTask = nil end
    if idleTrack1 then idleTrack1:Stop(0.2) end
    if idleTrack2 then idleTrack2:Stop(0.2) end
    for _, track in pairs(loadedManualEmotes) do
        track:Stop(0.2)
    end
end

-- Idle Sequence Logic
local function startIdleSequence()
    if isIdle or not isCustomAnimActive or isManualEmotePlaying then return end
```

```
    local char = player.Character
    if char and char:FindFirstChild("Humanoid") then
        local state = char.Humanoid:GetState()
        if isSpecialMovementState(state) then return end
    end

    isIdle = true
    if idleSequenceTask then task.cancel(idleSequenceTask) end

    idleSequenceTask = task.spawn(function()
        while isIdle and isCustomAnimActive do
            if isManualEmotePlaying then task.wait(0.5) continue end
            if idleTrack1 and not idleTrack1.IsPlaying then idleTrack1:Play(0.3) end

            local waitTime = (idleTrack1 and idleTrack1.Length > 0) and idleTrack1.Length or 5
            task.wait(waitTime)

            if not isIdle or isManualEmotePlaying then break end

            if math.random(1, 100) <= 20 then
                if idleTrack1 then idleTrack1:Stop(0.3) end
                if idleTrack2 then
                    idleTrack2:Play(0.3)
                    task.wait(99999999999999999)
                    if not isIdle or isManualEmotePlaying then break end
                    idleTrack2:Stop(0.3)
                end
            end
        end
    end)
end

-- Apply Animations
local function applyCustomAnimation(character)
    local humanoid = character:WaitForChild("Humanoid", 5)
    local animator = humanoid:WaitForChild("Animator", 5)
    if not animator then return end

    for _, conn in pairs(charConnections) do conn:Disconnect() end
    charConnections = {}
    currentMovementState = "Stopped"

    originalC0s = {}
    local descendants = character:GetDescendants()
    for i = 1, #descendants do
        local desc = descendants[i]
        if desc:IsA("Motor6D") then
            originalC0s[desc] = desc.C0
```

```
        end
    end

    local rAnim = Instance.new("Animation"); rAnim.AnimationId =
getTrueAnimationId(customRunId)
    customRunTrack = animator:LoadAnimation(rAnim); customRunTrack.Priority =
Enum.AnimationPriority.Action3

    local wAnim = Instance.new("Animation"); wAnim.AnimationId =
getTrueAnimationId(customWalkId)
    customWalkTrack = animator:LoadAnimation(wAnim); customWalkTrack.Priority =
Enum.AnimationPriority.Action3

    local a1 = Instance.new("Animation"); a1.AnimationId = getTrueAnimationId(customIdle1Id)
    idleTrack1 = animator:LoadAnimation(a1); idleTrack1.Priority =
Enum.AnimationPriority.Action2

    local a2 = Instance.new("Animation"); a2.AnimationId = getTrueAnimationId(customIdle2Id)
    idleTrack2 = animator:LoadAnimation(a2); idleTrack2.Priority =
Enum.AnimationPriority.Action2

    loadedManualEmotes = {}
    for _, data in ipairs(emoteData) do
        local anim = Instance.new("Animation")
        anim.AnimationId = getTrueAnimationId(data.ID)
        local track = animator:LoadAnimation(anim)
        track.Priority = Enum.AnimationPriority.Action4
        loadedManualEmotes[data.Name] = track
    end

    table.insert(charConnections, RunService.Heartbeat:Connect(function()
        if not isCustomAnimActive then return end

        local state = humanoid:GetState()
        local isSpecialState = isSpecialMovementState(state)

        if not isSpecialState then
            for motor, c0 in pairs(originalC0s) do
                if motor.Parent and (motor.Name:match("Shoulder") or motor.Name == "Waist" or
motor.Name == "Neck") then
                    motor.C0 = c0
                end
            end
        end

        if isSpecialState then
            if currentMovementState ~= "Special" then
                currentMovementState = "Special"
```

```
            if customWalkTrack then customWalkTrack:Stop(0.2) end
            if customRunTrack then customRunTrack:Stop(0.2) end
            stopAllManualTracks()
        end
        return
    end

    local isMoving = humanoid.MoveDirection.Magnitude > 0.05

    if isMoving then
        local velocity = humanoid.RootPart and humanoid.RootPart.Velocity or
Vector3.new(0,0,0)
        local flatSpeed = Vector3.new(velocity.X, 0, velocity.Z).Magnitude
        local animSpeed = math.max(flatSpeed, 2)

        local directionMultiplier = 1
        if humanoid.RootPart then
            local moveDir = humanoid.MoveDirection
            local lookDir = humanoid.RootPart.CFrame.LookVector
            if moveDir:Dot(lookDir) < -0.1 then
                directionMultiplier = -1
            end
        end

        if flatSpeed <= 10 then
            if currentMovementState ~= "Walking" then
                currentMovementState = "Walking"
                stopAllManualTracks()
                if customRunTrack then customRunTrack:Stop(0.2) end
                if customWalkTrack then customWalkTrack:Play(0.2) end
            end
            if customWalkTrack then customWalkTrack:AdjustSpeed((animSpeed / 10) *
directionMultiplier) end
        else
            if currentMovementState ~= "Running" then
                currentMovementState = "Running"
                stopAllManualTracks()
                if customWalkTrack then customWalkTrack:Stop(0.2) end
                if customRunTrack then customRunTrack:Play(0.2) end
            end
            if customRunTrack then customRunTrack:AdjustSpeed((animSpeed / 16) *
directionMultiplier) end
        end
    else
        if currentMovementState ~= "Stopped" then
            currentMovementState = "Stopped"
            if customWalkTrack then customWalkTrack:Stop(0.2) end
            if customRunTrack then customRunTrack:Stop(0.2) end
```

```
            end
            if not isManualEmotePlaying then startIdleSequence() end
        end
    end))
end

local function restoreOriginalAnimation(character)
    isCustomAnimActive = false

    for _, conn in pairs(charConnections) do conn:Disconnect() end
    charConnections = {}
    originalC0s = {}

    if customWalkTrack then customWalkTrack:Stop(0.2) end
    if customRunTrack then customRunTrack:Stop(0.2) end
    stopAllManualTracks()
end

-- Create Emote Buttons in Scroll
for _, data in ipairs(emoteData) do
    local btn = Instance.new("TextButton")
    btn.Name = data.Name
    btn.Parent = scroll
    btn.Size = UDim2.new(1, -5, 0, 30)
    btn.BackgroundColor3 = Color3.fromRGB(60, 60, 60)
    btn.Text = data.Name
    btn.TextColor3 = Color3.fromRGB(255, 255, 255)
    btn.TextScaled = true

    btn.MouseButton1Click:Connect(function()
        local char = player.Character
        if not char or not isCustomAnimActive then return end
        local hum = char:FindFirstChild("Humanoid")

        if hum then
            local state = hum:GetState()
            if hum.MoveDirection.Magnitude < 0.1 and not isSpecialMovementState(state) then
                stopAllManualTracks()
                isManualEmotePlaying = true
                local track = loadedManualEmotes[data.Name]
                if track then
                    track:Play(0.3)
                    local conn; conn = track.Stopped:Connect(function()
                        isManualEmotePlaying = false
                        if hum.MoveDirection.Magnitude < 0.1 then startIdleSequence() end
                        conn:Disconnect()
                    end)
                end
```

```
        end
    end
  end)
end

-- Variáveis para o sistema de Interação
local currentInteractTrack = nil
local interactLoop = nil

-- Create Interact Buttons in Scroll
for _, data in ipairs(interactData) do
  local btn = Instance.new("TextButton")
  btn.Name = data.Name
  btn.Parent = interactScroll
  btn.Size = UDim2.new(1, -5, 0, 30)
  btn.BackgroundColor3 = Color3.fromRGB(60, 60, 60)
  btn.Text = data.Name
  btn.TextColor3 = Color3.fromRGB(255, 255, 255)
  btn.TextScaled = true

  btn.MouseButton1Click:Connect(function()
    if data.AnimID then
      if player.Backpack:FindFirstChild(data.Name) or (player.Character and
player.Character:FindFirstChild(data.Name)) then
        return
      end

      local tool = Instance.new("Tool")
      tool.Name = data.Name
      tool.RequiresHandle = false
      tool.Parent = player.Backpack

      local mouse = player:GetMouse()
      local clickConn

      -- Configurações de velocidade e distância específicas para cada ação
      local speed = data.Speed or 15
      local baseZ = data.BaseZ or -2.2
      local amplitude = data.Amplitude or 0.3

      tool.Equipped:Connect(function()
        clickConn = tool.Activated:Connect(function()
          local target = mouse.Target
          if target and target.Parent and target.Parent:FindFirstChild("Humanoid") and
target.Parent ~= player.Character then
            local targetChar = target.Parent
            local myChar = player.Character
```

```
                if myChar and myChar:FindFirstChild("HumanoidRootPart") and
targetChar:FindFirstChild("HumanoidRootPart") then

                    if interactLoop then interactLoop:Disconnect(); interactLoop = nil end
                    if currentInteractTrack then currentInteractTrack:Stop() end

                    local animator = myChar.Humanoid:FindFirstChild("Animator")
                    if animator then
                        local anim = Instance.new("Animation")
                        anim.AnimationId = getTrueAnimationId(data.AnimID)
                        currentInteractTrack = animator:LoadAnimation(anim)
                        currentInteractTrack.Priority = Enum.AnimationPriority.Action4
                        currentInteractTrack:Play()
                    end

                    interactLoop = RunService.Heartbeat:Connect(function()
                        if not myChar or not targetChar or not
myChar:FindFirstChild("HumanoidRootPart") or not
targetChar:FindFirstChild("HumanoidRootPart") then
                            if interactLoop then interactLoop:Disconnect(); interactLoop = nil end
                            return
                        end

                        -- Usa as configurações específicas definidas na tabela interactData
                        local moveOffset = math.sin(tick() * speed) * amplitude

                        -- Posicionamento usando o BaseZ (quão longe) e moveOffset (o vaivém)
                        myChar.HumanoidRootPart.CFrame =
targetChar.HumanoidRootPart.CFrame * CFrame.new(0, 0, baseZ + moveOffset) *
CFrame.Angles(0, math.rad(180), 0)
                    end)
                end
            end
        end)

        tool.Unequipped:Connect(function()
            if clickConn then clickConn:Disconnect() end
            if interactLoop then
                interactLoop:Disconnect()
                interactLoop = nil
            end
            if currentInteractTrack then
                currentInteractTrack:Stop()
                currentInteractTrack = nil
            end
        end)
    else
```

```
            print("Clicou em: " .. data.Name)
        end
    end)
end

-- UI Toggles
emoteToggleButton.MouseButton1Click:Connect(function()
    emoteFrame.Visible = not emoteFrame.Visible
end)

interactButton.MouseButton1Click:Connect(function()
    interactFrame.Visible = not interactFrame.Visible
end)

onButton.MouseButton1Click:Connect(function()
    if not isCustomAnimActive then
        isCustomAnimActive = true
        statusLabel.Text = "Status: Online"; statusLabel.TextColor3 = Color3.fromRGB(0, 255, 0)
        if player.Character then applyCustomAnimation(player.Character) end
    end
end)

offButton.MouseButton1Click:Connect(function()
    if isCustomAnimActive then
        statusLabel.Text = "Status: Offline"; statusLabel.TextColor3 = Color3.fromRGB(255, 0, 0)
        if player.Character then restoreOriginalAnimation(player.Character) end
    end
end)

destroyButton.MouseButton1Click:Connect(function()
    if isCustomAnimActive and player.Character then
        restoreOriginalAnimation(player.Character)
    end
    screenGui:Destroy()
end)

player.CharacterAdded:Connect(function(c)
    if isCustomAnimActive then task.wait(0.5); applyCustomAnimation(c) end
end)
```